

Advanced Python Iterators, Generators, Decorators & Context Managers

1. Introduction to Iteration

Iteration means processing elements one at a time from a collection.

```
numbers = [10, 20, 30]
for num in numbers:
    print(num)
```

2. Iterable vs Iterato

Iterable

An object that can be traversed and can create an iterator.

Common Iterables:
(list,tuple,set,dictionary,string)

Iterator

An object that produces items one at a time.

Must implement:
__iter__()
__next__()

Difference Between Iterable and Iterator

Difference Between Iterable and Iterator

Iterable	Iterator
Collection of data	Object used to traverse data
Uses <code>__iter__()</code>	Uses both <code>__iter__()</code> and <code>__next__()</code>
Cannot directly use <code>next()</code>	Can use <code>next()</code>
Example: list, tuple, string	Example: output of <code>iter(list)</code>

3. iter()

`iter()` converts an iterable into an iterator.
Syntax: iterator = iter(iterable)

4. next()

Retrieves the next element from an iterator.
Example:

```
nums = [100, 200, 300]
it = iter(nums)
print(next(it))
```

5. StopIteration

Raised automatically when all elements are exhausted.

Example:

```
nums = [1, 2]
it = iter(nums)
print(next(it))
print(next(it))
print(next(it))
```

6. Custom Iterator Class

To create a custom iterator:

Step 1

Implement `__iter__()`

Step 2

Implement `__next__()`

Example:

class Counter:

```
    def __init__(self):
        self.current = 1
    def __iter__(self):
        return self
    def __next__(self):
        if self.current > 5:
            raise StopIteration
        value = self.current
        self.current += 1
        return value
```

7. Number Iterator

Generates numbers from start to end.

Implementation

class NumberIterator:

```
    def __init__(self, start, end):
        self.current = start
        self.end = end
    def __iter__(self):
        return self
    def __next__(self):
        if self.current > self.end:
            raise StopIteration
```

```
value = self.current
self.current += 1
return va
```

Performance Benchmark

Comparison Report (Expected Findings)

1. List vs Generator

- **Execution Time:** Lists are often slightly faster for repeated access because data is precomputed.
- **Memory Usage:** Generators use significantly less memory since values are produced on demand.
- **Winner:** Generator (memory efficiency).

2. Normal Function vs Decorator

- **Execution Time:** Decorated functions incur a small wrapper-call overhead.
- **Memory Usage:** Nearly identical.
- **Winner:** Normal Function (slightly faster).

3. Manual File Handling vs Context Manager

- **Execution Time:** Usually very similar.
- **Memory Usage:** Similar.
- **Winner:** Context Manager (`with`) because it is safer, cleaner, and automatically closes files.

4. Iterator vs List Iteration

- **Execution Time:** Comparable, depending on workload.
- **Memory Usage:** Iterators consume far less memory because they do not store the entire collection.
- **Winner:** Iterator (memory efficiency).